

Implementarea unui workflow multi-agent în Microsoft Azure AI Foundry pentru automatizarea proiectării aplicațiilor software

1. Introducere

În ultimii ani, inteligența artificială a evoluat rapid de la simple modele generative către sisteme complexe bazate pe servicii cloud și agenți autonomi. Conceptul de AI as a Service (AlaaS) oferă dezvoltatorilor acces la modele și infrastructură AI prin intermediul platformelor cloud, fără necesitatea administrării hardware-ului propriu.

Inițial, sistemele AI erau utilizate în principal pentru generare de text sau cod, având un comportament reactiv. Instrumente precum GitHub Copilot, ChatGPT sau Gemini răspundeau la solicitările utilizatorului fără a coordona procese complexe. Începând cu anul 2026, industria software se orientează către conceptul de Agentic AI, unde agenții AI pot planifica acțiuni, utiliza instrumente externe și colabora între ei pentru rezolvarea unor sarcini complexe.

Această tranziție schimbă și rolul dezvoltatorului software. Accentul nu mai este pus exclusiv pe scrierea manuală a codului, ci pe definirea specificațiilor și supervizarea fluxurilor de lucru automatizate. Astfel apare conceptul de „Spec-First Workflow”, în care dezvoltatorul definește obiectivul aplicației, iar agenții AI generează automat pașii necesari implementării și verificării acesteia.

Marile platforme cloud, precum Microsoft Azure AI Foundry, AWS Bedrock Agents sau Google Vertex AI, oferă suport pentru dezvoltarea de sisteme multi-agent și workflow-uri automatizate. Aceste platforme permit orchestrarea agenților AI, integrarea modelelor și automatizarea proceselor software.

În cadrul acestui referat este analizată platforma Microsoft Azure AI Foundry și este prezentat un experiment propriu bazat pe un workflow multi-agent pentru automatizarea procesului de proiectare software. Experimentul urmărește transformarea unei idei software într-o specificație completă folosind agenți AI specializați pentru analiză, arhitectură și verificare QA.

2. AlaaS și Agentic AI

2.1 Ce este AlaaS

AI as a Service (AlaaS) reprezintă furnizarea capabilităților de inteligență artificială prin intermediul platformelor cloud. Prin acest model, dezvoltatorii și organizațiile pot utiliza modele AI, infrastructură și servicii specializate fără a administra direct resurse hardware complexe.

Platformele AlaaS oferă acces la modele de generare text, analiză de imagini, procesare de limbaj natural și automatizare software prin API-uri și servicii cloud scalabile. Printre cele mai utilizate platforme AlaaS se află Microsoft Azure AI Foundry, AWS Bedrock și Google Vertex AI.

2.2 Evoluția către Agentic AI

Modelele AI generative tradiționale funcționează în principal într-un mod reactiv, oferind răspunsuri la solicitările utilizatorului fără a coordona procese complexe sau fluxuri de lucru autonome.

Conceptul de Agentic AI introduce agenți capabili să planifice acțiuni, să utilizeze instrumente externe și să colaboreze cu alți agenți specializați pentru rezolvarea unor sarcini complexe. În loc să genereze doar conținut, agenții AI pot executa secvențe de task-uri și pot adapta procesul în funcție de rezultatele intermediare.

Un sistem agentic modern poate include:

- agenți pentru analiză de cerințe;
- agenți pentru generare de arhitectură software;
- agenți QA pentru verificarea calității;
- mecanisme de orchestration între agenți.

Această abordare permite automatizarea unor procese software complexe și reduce timpul necesar proiectării și implementării aplicațiilor.

2.3 Spec-First Workflow

Conceptul de „Spec-First Workflow” apare tot mai frecvent în dezvoltarea software modernă, unde accentul este pus pe definirea specificațiilor și obiectivelor aplicației înaintea implementării efective a codului. În acest model, agenții AI utilizează cerințele furnizate pentru a genera automat planuri de implementare, arhitecturi software, analize QA și recomandări tehnice. Astfel, dezvoltatorul devine un coordonator al fluxului de lucru și supervisor al agenților AI.

Procesul unui workflow de tip Agentic AI începe prin definirea unei cerințe generale de către dezvoltator sau utilizator. Cerința reprezintă obiectivul principal al aplicației și poate fi formulată într-un mod simplu sau incomplet.

Prima etapă constă în analiza cerinței de către un agent specializat în clarificarea și optimizarea specificațiilor. Rolul acestuia este de a identifica obiectivul aplicației, utilizatorii principali,

funcționalitățile așteptate și contextul tehnic necesar.

După clarificarea cerinței, informațiile sunt transmise către un agent responsabil de analiza produsului software. Acesta generează cerințele funcționale, cerințele non-funcționale și user stories necesare dezvoltării aplicației.

Etapă următoare este realizată de un agent de arhitectură software, care proiectează structura aplicației, stabilește microserviciile necesare, bazele de date utilizate și modul de comunicare dintre componentele sistemului.

Arhitectura generată este apoi analizată de un agent specializat în Quality Assurance și analiză de risc. Acesta identifică posibile probleme de securitate, edge cases, riscuri de performanță și generează scenarii de testare pentru verificarea aplicației.

Rezultatul final al workflow-ului constă într-o specificație software completă, generată automat prin colaborarea dintre agenții AI specializați.

3. Microsoft Azure AI Foundry

3.1 Ce este Microsoft Azure AI Foundry

Microsoft Azure AI Foundry reprezintă o platformă cloud destinată dezvoltării, orchestrării și implementării aplicațiilor bazate pe inteligență artificială. Platforma oferă suport pentru integrarea modelelor AI, construirea de agenți autonomi și crearea de workflow-uri multi-agent pentru automatizarea proceselor software.

Azure AI Foundry permite dezvoltatorilor să utilizeze modele AI prin intermediul serviciilor cloud, fără a administra direct infrastructura hardware. Platforma oferă funcționalități pentru deployment de modele, monitorizare, evaluare și colaborare între agenți AI specializați.

Unul dintre principalele avantaje ale platformei este suportul pentru dezvoltarea sistemelor de tip Agentic AI, unde mai mulți agenți pot colabora în cadrul unui workflow orchestrat pentru rezolvarea unor sarcini complexe.

3.2 Componentele platformei

Agents

Agenții reprezintă componente AI specializate care execută task-uri specifice în cadrul unui workflow. Fiecare agent poate avea instrucțiuni proprii, roluri diferite și acces la modele AI distincte.

În cadrul experimentului realizat au fost utilizați mai mulți agenți specializați pentru optimizarea cerințelor, analiza produsului software, proiectarea arhitecturii și analiza QA.

Workflows

Workflows permit orchestrarea secvențială a agenților AI și automatizarea fluxului de lucru dintre aceștia. Outputul generat de un agent poate fi transmis automat următorului agent din workflow, fără

intervenție manuală.

În experimentul realizat a fost utilizat un workflow secvențial pentru transmiterea automată a informațiilor între agenții AI specializați.

Model Deployments

Platforma permite deployment-ul modelelor AI direct în cloud și utilizarea acestora în cadrul agenților și workflow-urilor. Modelele pot fi configurate și reutilizate în mai multe componente ale aplicației.

3.3 Utilizarea platformei în experiment

În cadrul experimentului a fost creat un proiect nou în Microsoft Azure AI Foundry și a fost configurat un workflow multi-agent destinat automatizării procesului de proiectare software.

Au fost definiți mai mulți agenți specializați:

- PromptOptimizationAgent;
- ProductAnalystAgent;
- SoftwareArchitectAgent;
- QAandRiskAgent.

Agenții au fost conectați prin intermediul unui workflow secvențial, astfel încât rezultatul fiecărui agent să fie transmis automat către următorul agent din flux.

Pentru rularea workflow-ului a fost utilizat un model AI deployat în platformă, iar întregul proces a fost executat automat pornind de la o singură cerință introdusă de utilizator.

4. Experiment

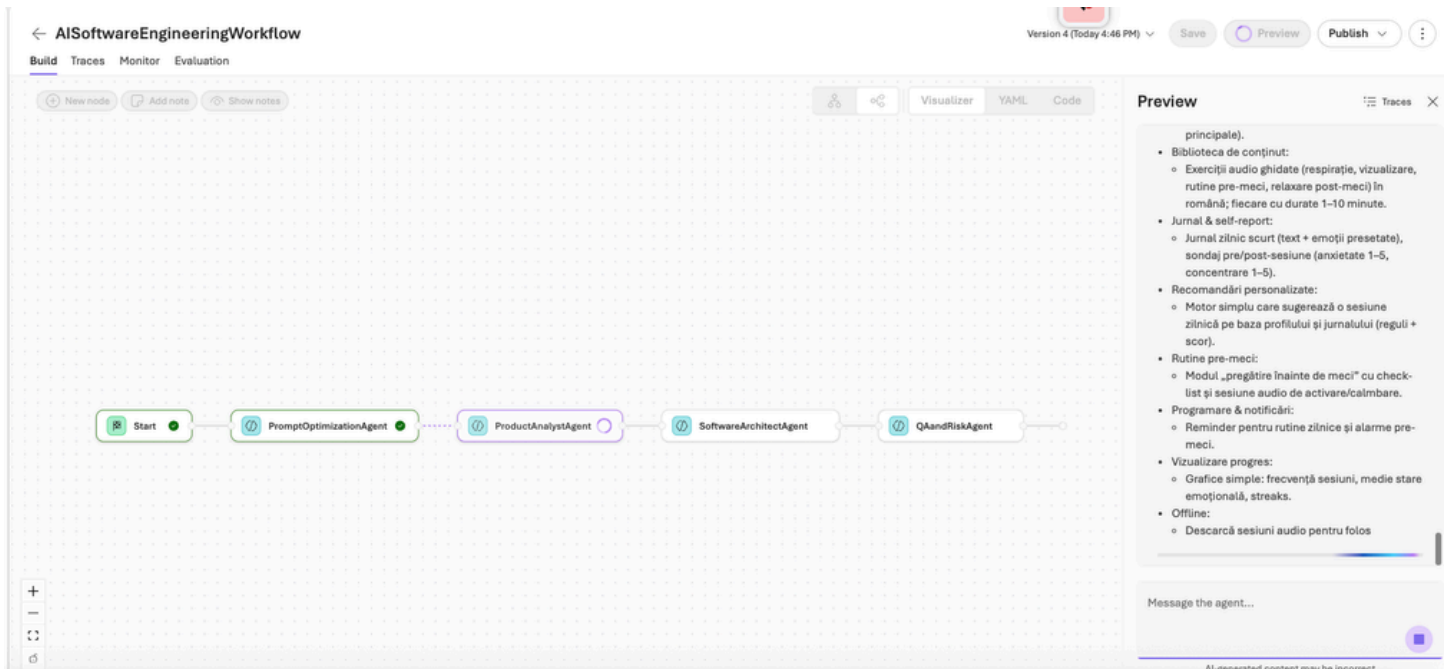
4.1 Scopul experimentului

Scopul experimentului realizat a fost construirea unui workflow multi-agent capabil să transforme automat o idee software formulată într-un mod general într-o specificație software completă. Experimentul urmărește simularea unui proces modern de dezvoltare software bazat pe colaborarea dintre agenți AI specializați.

Workflow-ul creat utilizează mai mulți agenți autonomi care colaborează secvențial pentru clarificarea cerințelor, analiza produsului software, proiectarea arhitecturii și verificarea calității soluției generate.

Soluția implementată demonstrează modul în care platformele AlaaS moderne pot automatiza etape importante ale procesului de proiectare software și pot reduce intervenția manuală necesară în fazele inițiale ale dezvoltării.

4.2 Arhitectura workflow-ului



Workflow-ul implementat utilizează o arhitectură secvențială, în care outputul fiecărui agent este transmis automat către următorul agent din flux. Procesul începe prin introducerea unei cerințe generale de către utilizator, iar fiecare agent execută o etapă specializată a procesului de analiză și proiectare software.

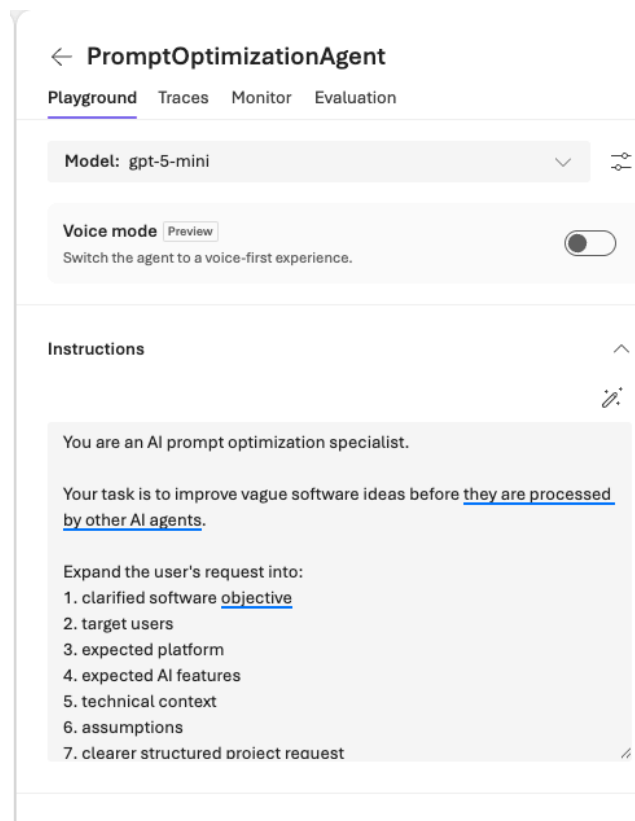
Această abordare permite separarea responsabilităților între agenți și simularea unui proces colaborativ asemănător echipelor reale de dezvoltare software.

4.3 Descrierea agenților

4.3.1 PromptOptimizationAgent

PromptOptimizationAgent are rolul de a clarifica și restructura cerința inițială introdusă de utilizator. Agentul identifică obiectivul aplicației, tipul utilizatorilor și contextul tehnic necesar pentru etapele următoare ale workflow-ului.

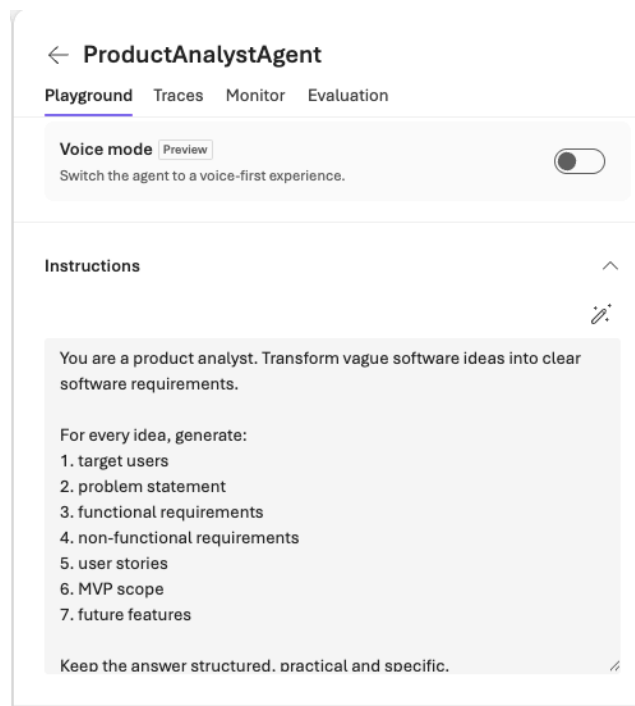
Rezultatul generat de acest agent este utilizat ca input pentru analiza produsului software.



4.3.2 ProductAnalystAgent

ProductAnalystAgent este responsabil de generarea cerințelor funcționale și non-funcționale ale aplicației. Agentul produce user stories, funcționalități principale și o descriere generală a produsului software.

Acest agent simulează rolul unui product analyst dintr-o echipă reală de dezvoltare software.

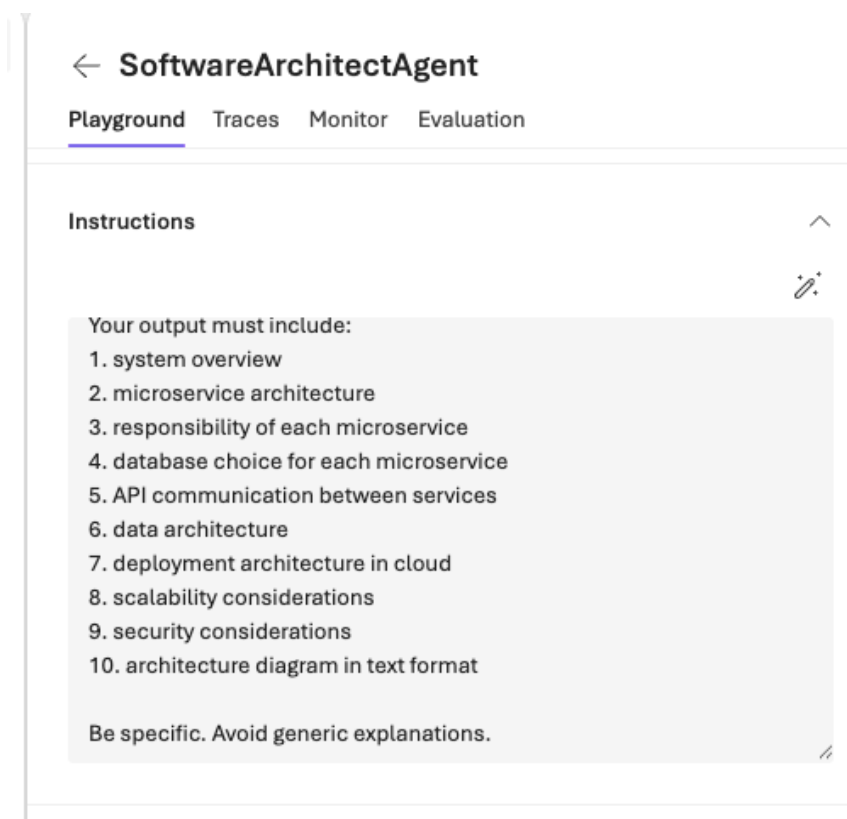


4.3.3 SoftwareArchitectAgent

SoftwareArchitectAgent generează arhitectura tehnică a aplicației pe baza cerințelor definite anterior. Agentul stabilește structura aplicației, microserviciile necesare, bazele de date și modul de

comunicare dintre componente.

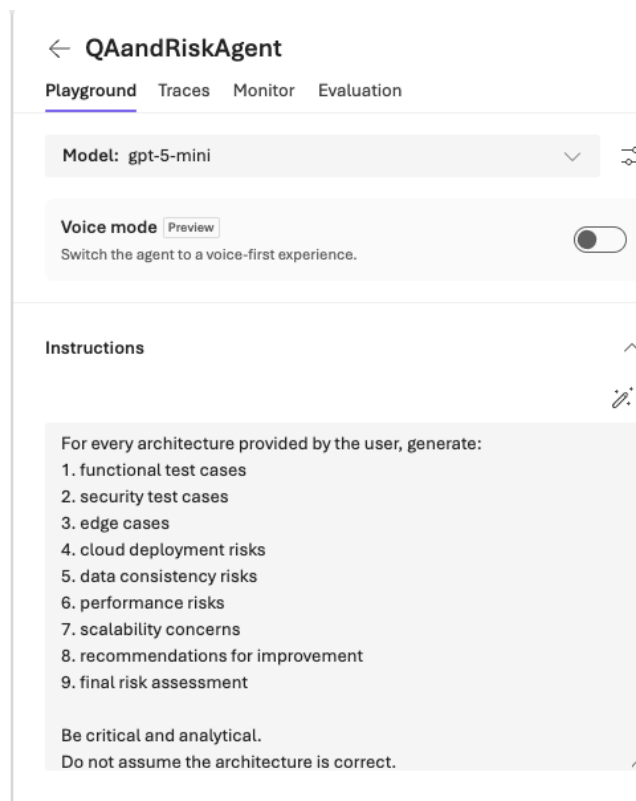
Rezultatul obținut reprezintă o arhitectură software completă care poate fi utilizată ca bază pentru implementarea aplicației.



4.3.4 QAandRiskAgent

QAandRiskAgent analizează arhitectura generată și identifică posibile probleme de securitate, edge cases și riscuri de performanță. Agentul generează scenarii de testare și recomandări pentru îmbunătățirea soluției.

Acest agent simulează procesul de verificare și Quality Assurance utilizat în dezvoltarea software modernă.



4.4 Implementarea în Azure AI Foundry

Implementarea experimentului a fost realizată utilizând platforma Microsoft Azure AI Foundry. În cadrul platformei a fost creat un proiect nou destinat dezvoltării unui workflow multi-agent pentru automatizarea etapelor de analiză și proiectare software.

Primul pas al implementării a constat în configurarea proiectului și deployarea unui model AI utilizat de agenții workflow-ului. Modelul deployat a fost configurat pentru procesarea cerințelor utilizatorului și generarea răspunsurilor necesare fiecărei etape din flux.

După configurarea proiectului, au fost creați mai mulți agenți specializați, fiecare având un rol distinct în cadrul workflow-ului:

- PromptOptimizationAgent;
- ProductAnalystAgent;
- SoftwareArchitectAgent;
- QAandRiskAgent.

Fiecare agent a fost configurat utilizând instrucțiuni specifice („Instructions”) pentru definirea comportamentului și a responsabilităților sale în cadrul procesului.

Pentru automatizarea colaborării dintre agenți a fost utilizat un workflow de tip „Sequential Workflow”. Acest tip de workflow permite transmiterea automată a outputului generat de un agent către următorul agent din flux, fără intervenție manuală.

Procesul începe prin introducerea unei cerințe generale de către utilizator. Cerința este analizată și optimizată de primul agent, iar rezultatul este transmis secvențial către ceilalți agenți pentru generarea cerințelor software, proiectarea arhitecturii și analiza QA.

Implementarea realizată demonstrează modul în care platformele AlaaS moderne pot fi utilizate pentru orchestrarea unui sistem multi-agent și automatizarea etapelor specifice procesului de dezvoltare software.

4.5 Rezultatele experimentului

În urma implementării workflow-ului multi-agent în Microsoft Azure AI Foundry, sistemul a reușit să proceseze automat cerința introdusă de utilizator și să genereze o specificație software completă utilizând colaborarea dintre agenții AI specializați.

Workflow-ul a început prin introducerea unei cerințe generale pentru dezvoltarea unei aplicații AI destinate monitorizării performanței mentale și progresului sportivilor de tenis. Cerința a fost analizată inițial de PromptOptimizationAgent, care a clarificat obiectivul aplicației și a restructurat cerința într-o formă mai detaliată și mai ușor de procesat.

Rezultatul obținut a fost transmis automat către ProductAnalystAgent, care a generat cerințe funcționale, cerințe non-funcționale și user stories pentru aplicația propusă. Agentul a identificat funcționalități precum monitorizarea stării mentale, jurnalul post-meci, analiza progresului și recomandările AI pentru utilizatori.

În etapa următoare, SoftwareArchitectAgent a generat arhitectura tehnică a aplicației. Agentul a propus utilizarea unei arhitecturi bazate pe microservicii și a stabilit componentele principale ale sistemului, bazele de date necesare și comunicarea dintre servicii.

Ultima etapă a workflow-ului a fost realizată de QAandRiskAgent, care a analizat arhitectura generată și a identificat posibile riscuri legate de securitate, performanță și scalabilitate. De asemenea, agentul a generat scenarii de testare și recomandări pentru îmbunătățirea soluției.

Rezultatele obținute demonstrează faptul că un workflow multi-agent poate automatiza mai multe etape ale procesului de proiectare software și poate produce rezultate coerente pornind de la o singură cerință introdusă de utilizator.

Un aspect important observat în cadrul experimentului a fost transmiterea automată a contextului între agenți prin intermediul workflow-ului secvențial configurat în Azure AI Foundry. Această abordare a permis colaborarea agenților fără intervenție manuală între etape și a simulat un proces real de dezvoltare software bazat pe orchestrarea agenților AI.

ProductAnalystAgent a identificat principalele categorii de utilizatori ai aplicației, inclusiv jucători recreaționali, antrenori și psihologi sportivi. De asemenea, agentul a generat cerințe funcționale pentru onboarding, sesiuni audio ghidate, jurnal mental și dashboard de progres.

Exemple de funcționalități generate:

- jurnal text și voice;
- sesiuni mindfulness pre-meci;
- notificări și memento-uri;

- recomandări AI personalizate.

SoftwareArchitectAgent a propus o arhitectură bazată pe microservicii și servicii cloud gestionate. Arhitectura include servicii pentru autentificare, gestionarea conținutului audio, analiză AI, notificări și analytics.

Exemple de microservicii generate:

- Auth Service;
- Journal Service;
- AI/ML Service;
- Analytics & Reporting Service;
- Notifications Service.

QAandRiskAgent a identificat mai multe riscuri legate de securitate, sincronizarea datelor offline și costurile serviciilor AI. De asemenea, agentul a generat scenarii de testare pentru autentificare, upload de fișiere audio, sincronizare offline și controlul accesului utilizatorilor.

Exemple de riscuri identificate:

- expunerea datelor sensibile către servicii AI externe;
- conflicte de sincronizare offline;
- costuri ridicate generate de inferența AI;
- probleme de scalabilitate în cazul creșterii numărului de utilizatori.

4.6 Limitări observate

Pe parcursul implementării și testării workflow-ului multi-agent au fost observate mai multe limitări specifice sistemelor bazate pe inteligență artificială și orchestrarea agenților AI.

O primă limitare importantă este dependența ridicată de calitatea prompturilor utilizate. Rezultatele generate de agenți depind direct de claritatea și structura cerințelor introduse de utilizator. Prompturile incomplete sau ambigue pot conduce la răspunsuri incoerente sau insuficient detaliate.

De asemenea, în cadrul workflow-ului au fost observate situații în care informațiile transmise între agenți conțineau elemente redundante sau repetitive. Acest aspect apare frecvent în sistemele multi-agent și poate afecta claritatea rezultatului final.

O altă limitare identificată este posibilitatea apariției fenomenului de „hallucination”, specific modelelor generative AI. În anumite situații, agenții pot genera detalii tehnice incorecte sau recomandări care necesită validare umană suplimentară.

Complexitatea workflow-urilor multi-agent poate conduce și la creșterea timpului de procesare, mai ales atunci când sunt utilizate mai multe etape secvențiale și modele AI de dimensiuni mari. Acest lucru poate afecta performanța sistemului și costurile asociate utilizării serviciilor cloud.

În plus, utilizarea platformelor AlaaS ridică probleme legate de securitatea și confidențialitatea datelor, în special atunci când sunt procesate informații sensibile sau date personale ale utilizatorilor. Din acest motiv, este necesară implementarea unor mecanisme suplimentare de criptare, control al accesului și protecție a datelor.

Cu toate aceste limitări, experimentul realizat demonstrează potențialul semnificativ al sistemelor multi-agent și al platformelor AlaaS moderne pentru automatizarea etapelor de analiză și proiectare software.

5. Avantaje și dezavantaje

5.1 Avantaje

Utilizarea unui workflow multi-agent bazat pe platforme AlaaS moderne oferă numeroase avantaje în procesul de dezvoltare software.

Un prim avantaj important este automatizarea etapelor repetitive și consumatoare de timp. Agenții AI pot genera rapid cerințe software, arhitecturi tehnice și scenarii de testare fără intervenție manuală constantă.

Un alt avantaj este viteza ridicată de prototipare. Workflow-ul implementat permite transformarea unei idei generale într-o specificație software detaliată într-un timp foarte scurt comparativ cu procesele tradiționale.

Separarea responsabilităților între agenți specializați contribuie la organizarea mai clară a procesului software și la simularea unui workflow apropiat de modul de lucru al echipelor reale de dezvoltare.

Platformele AlaaS moderne oferă și scalabilitate ridicată, acces rapid la modele AI performante și posibilitatea integrării mai multor servicii cloud într-un singur workflow orchestrat.

De asemenea, utilizarea agenților AI poate reduce efortul necesar etapelor inițiale de analiză și documentare și poate accelera procesul de luare a deciziilor tehnice.

5.2 Dezavantaje

În ciuda avantajelor oferite, utilizarea workflow-urilor multi-agent bazate pe inteligență artificială prezintă și o serie de dezavantaje și limitări.

Unul dintre principalele dezavantaje este dependența ridicată de prompt engineering și de calitatea specificațiilor introduse de utilizator. Rezultatele generate de agenți pot varia semnificativ în funcție de claritatea cerințelor formulate.

Modelele AI pot genera informații incorecte sau incomplete, ceea ce impune validarea permanentă a rezultatelor de către dezvoltatori umani.

Utilizarea serviciilor AI în cloud poate genera costuri ridicate, în special în cazul workflow-urilor complexe sau al utilizării modelelor mari de inteligență artificială.

De asemenea, sistemele multi-agent introduc complexitate suplimentară în ceea ce privește orchestrarea, transmiterea contextului și monitorizarea fluxului de lucru dintre agenți.

Un alt aspect important îl reprezintă securitatea datelor și protecția informațiilor sensibile procesate de platformele AlaaS. Utilizarea serviciilor externe poate ridica probleme legate de confidențialitate și conformitate.

6. Concluzii

Evoluția platformelor AlaaS și apariția sistemelor de tip Agentic AI determină schimbări importante în domeniul dezvoltării software moderne. Inteligența artificială nu mai este utilizată exclusiv pentru generarea de conținut sau cod, ci începe să fie integrată în workflow-uri complexe bazate pe colaborarea dintre agenți specializați.

În cadrul acestui referat a fost analizată platforma Microsoft Azure AI Foundry și a fost implementat un workflow multi-agent destinat automatizării procesului de analiză și proiectare software. Sistemul realizat a demonstrat posibilitatea colaborării automate dintre agenți AI pentru generarea cerințelor software, proiectarea arhitecturii și analiza QA.

Experimentul efectuat evidențiază avantajele utilizării platformelor AlaaS moderne în accelerarea procesului de dezvoltare software și automatizarea etapelor repetitive. Totodată, au fost identificate și limitări legate de calitatea rezultatelor generate, dependența de prompturi și problemele de securitate și cost asociate utilizării serviciilor AI în cloud.

Rezultatele obținute sugerează faptul că viitorul dezvoltării software se va baza tot mai mult pe colaborarea dintre dezvoltatori și agenți AI specializați, în cadrul unor workflow-uri autonome și orchestrate. Rolul dezvoltatorului evoluează astfel de la simplu programator la arhitect și supervisor al sistemelor AI.